

MIST – Modular Integration of Substitution and Transposition

João Meyer Muhlmann e Vitor Faria Medeiros da Silveira

1 Cifra criada

A cifra criada foi denominada **MIST** (*Modular Integration of Substitution and Transposition*).

O algoritmo combina duas técnicas:

1. substituição baseada na cifra de Vigenère;
2. transposição utilizando Rail Fence.

A cifragem é realizada na seguinte ordem:

$$\text{Texto} \rightarrow \text{Vigenre} \rightarrow \text{RailFence} \rightarrow \text{Textocifrado}$$

Para decifrar, as operações são realizadas na ordem inversa:

$$\text{Textocifrado} \rightarrow \text{RailFence}^{-1} \rightarrow \text{Vigenre}^{-1} \rightarrow \text{Textooriginal}$$

1.1 Substituição

A cifra de Vigenère foi adaptada para trabalhar diretamente com os bytes do arquivo.

A chave fornecida pelo usuário é repetida até possuir o mesmo tamanho do arquivo.

Para cada byte do texto, a cifragem é realizada por:

$$c_i = (p_i + k_i) \bmod 256$$

onde p_i é o byte original, k_i é o byte correspondente da chave e c_i é o byte cifrado.

A decifragem utiliza:

$$p_i = (c_i - k_i + 256) \bmod 256$$

Foi utilizado módulo 256 porque um byte pode possuir valores entre 0 e 255.

Essa abordagem também permite trabalhar com arquivos UTF-8. Caracteres acentuados podem possuir mais de um byte, mas o algoritmo não precisa interpretar os caracteres: ele cifra e decifra individualmente os bytes do arquivo.

1.2 Transposição

Após a substituição é aplicada uma cifra Rail Fence com oito trilhos.

Os bytes são distribuídos entre os trilhos seguindo um movimento de zigue-zague. Ao final, o conteúdo de cada trilho é concatenado para formar o texto cifrado.

Na decifragem é calculada inicialmente a quantidade de bytes pertencente a cada trilho. A partir disso são determinadas as posições de início de cada trilho, permitindo reconstruir a sequência original.

2 Implementação

A implementação foi realizada em C++.

Os arquivos são abertos em modo binário:

```
ifstream entrada(nomeDoArquivo, ios::binary);
```

O conteúdo completo é armazenado em uma `string`. Apesar do uso de `string`, os valores utilizados nos cálculos da substituição são convertidos para `unsigned char`, garantindo valores entre 0 e 255.

A cifragem foi implementada pela função:

```
void encripta_jovi(string &s, string chave)
```

Essa função primeiro aplica a substituição Vigenère e depois chama a função de transposição Rail Fence.

A decifragem é realizada por:

```
void decrypta_jovi(string &s, string chave)
```

Nesse caso, primeiro é desfeita a transposição e depois é aplicada a operação inversa da substituição.

As principais funções implementadas foram:

- `geraChave`: repete a chave até o tamanho necessário;
- `zigzag`: aplica a transposição Rail Fence;
- `dezizzag`: desfaz a transposição;
- `encripta_jovi`: realiza a cifragem MIST;
- `decrypta_jovi`: realiza a decifragem MIST.

A leitura linha a linha não foi utilizada, pois o arquivo cifrado pode conter qualquer valor de byte, incluindo valores correspondentes a quebras de linha ou ao byte zero.

A correção da implementação foi testada cifrando e posteriormente decifrando um arquivo UTF-8. O arquivo recuperado apresentou o mesmo hash SHA-256 do arquivo original.

3 Comparação de desempenho

O MIST foi comparado com:

- AES-256-CBC;
- RSA-2048 com OAEP e SHA-256.

AES e RSA foram executados utilizando a biblioteca OpenSSL.
Foram utilizados quatro textos do Projeto Gutenberg:

Arquivo	Bytes	Faixa
Gettysburg Address	900	< 1 KB
The Raven	7.582	1–10 KB
Alice’s Adventures in Wonderland	174.311	100 KB–1 MB
War and Peace	3.359.610	> 1 MB

Tabela 1: Arquivos utilizados nos testes.

Para MIST e AES foram realizadas 15 execuções de cada operação, utilizando a mediana dos tempos. Para RSA foram realizadas 3 execuções, devido ao maior custo computacional.

A leitura e escrita dos arquivos não fizeram parte do tempo medido.

No RSA, como uma operação RSA-2048 com OAEP e SHA-256 permite cifrar no máximo 190 bytes, os arquivos foram divididos em blocos desse tamanho.

3.1 Resultados de cifragem

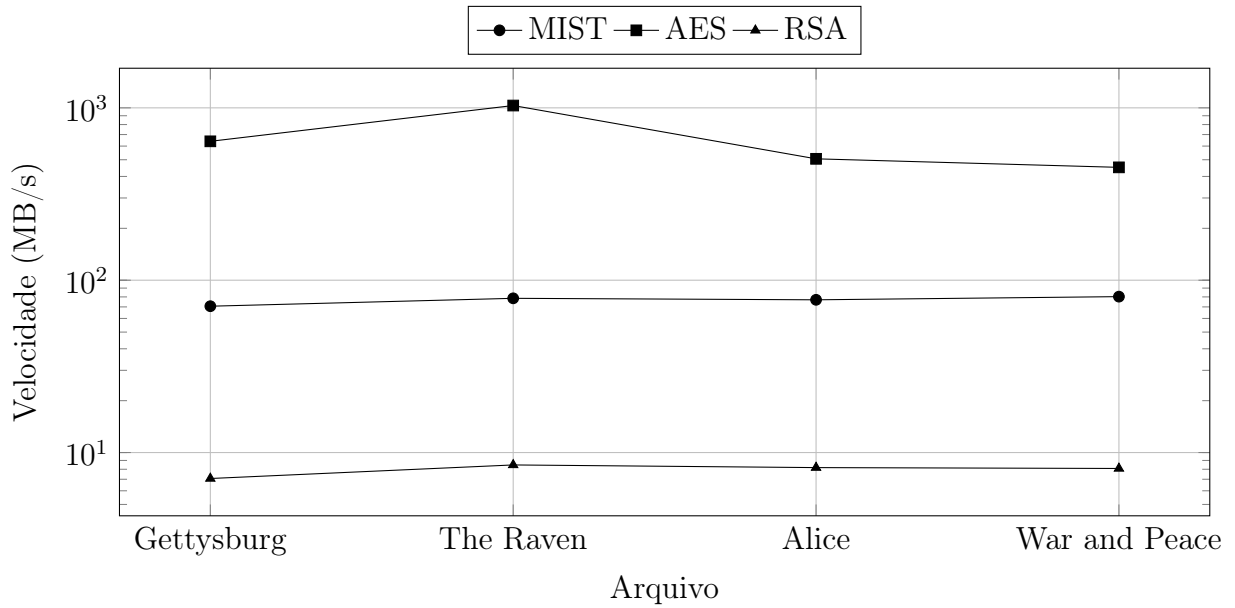


Figura 1: Velocidade de cifragem.

3.2 Resultados de decifragem

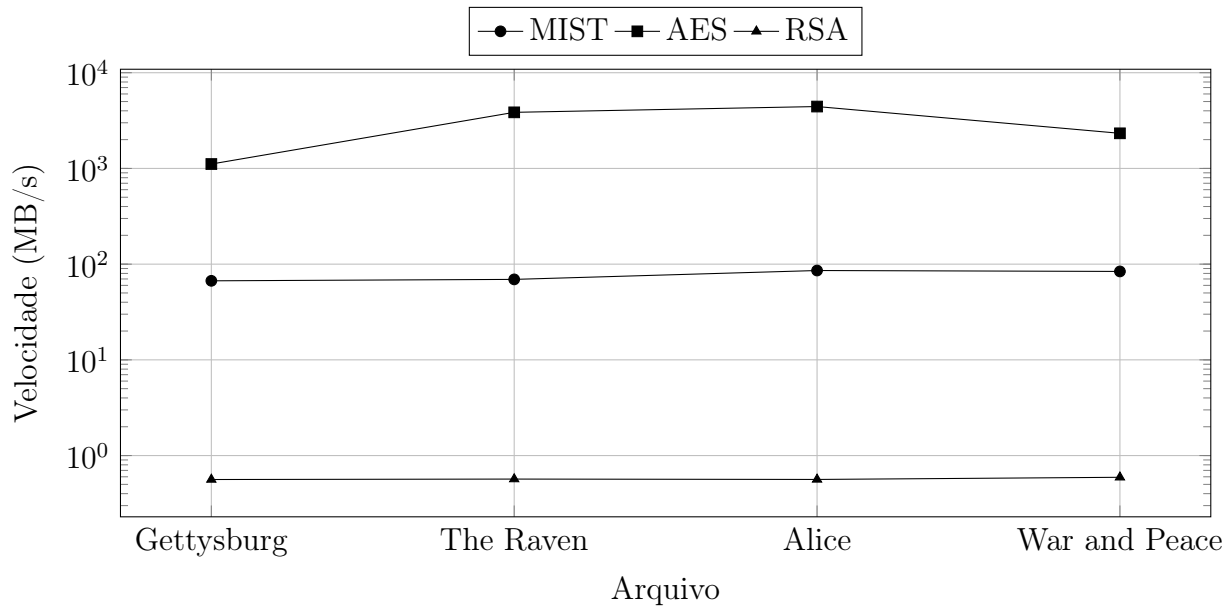


Figura 2: Velocidade de decifragem.

4 Conclusão

Nos testes realizados, o AES apresentou o maior desempenho tanto na cifragem quanto na decifragem.

O MIST apresentou velocidade relativamente estável para os arquivos maiores, próxima de 80 MB/s, ficando entre AES e RSA.

O RSA apresentou desempenho significativamente menor, principalmente na decifragem. Esse comportamento era esperado, pois RSA é um algoritmo assimétrico e não é normalmente utilizado para cifrar diretamente grandes volumes de dados.

O algoritmo MIST cumpriu o objetivo proposto de combinar uma técnica de substituição com uma técnica de transposição, permitindo cifrar e decifrar corretamente arquivos de texto UTF-8.